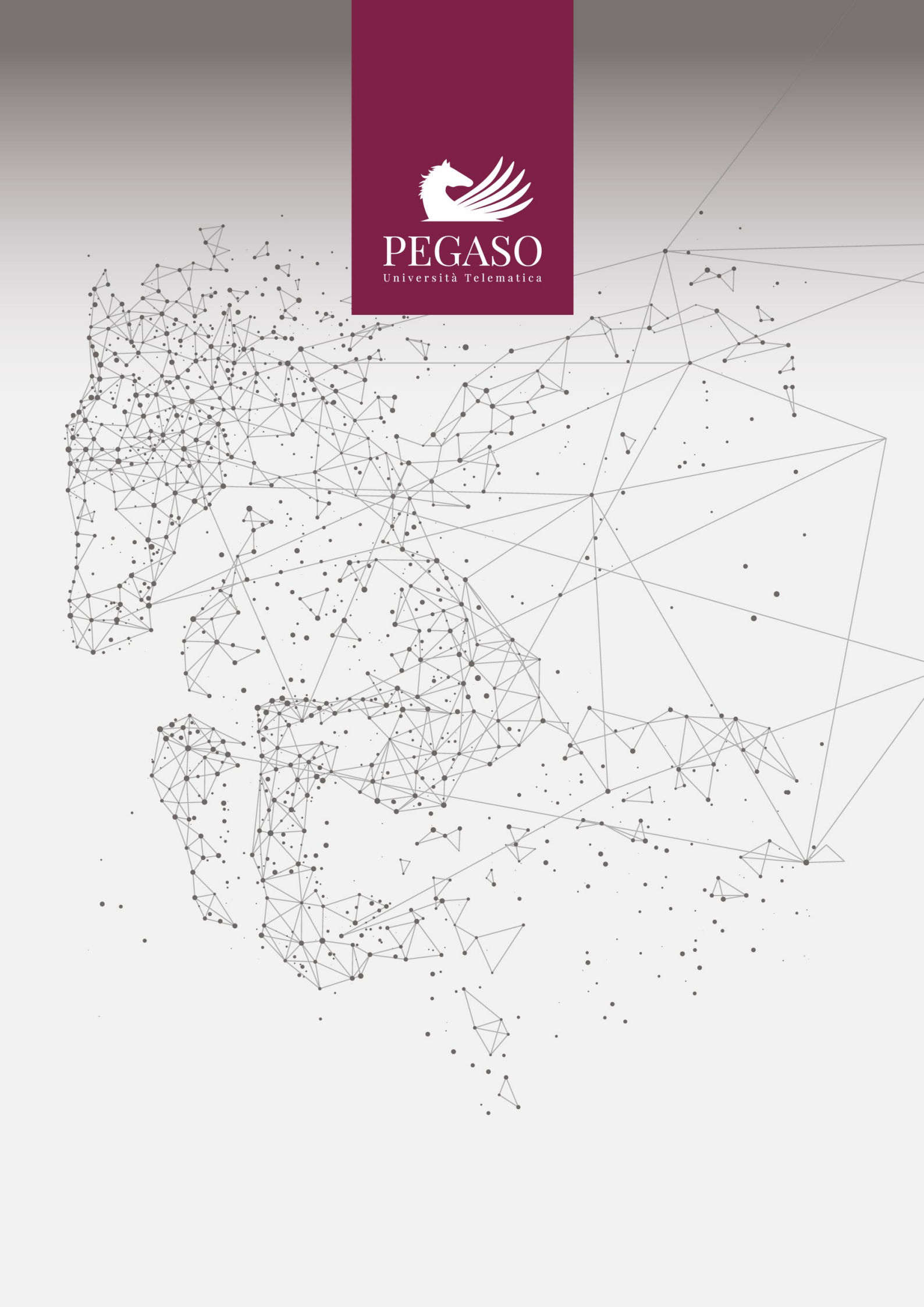

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. APERTURA 3
2. SCALABILITÀ 6
3. TECNICHE DI SCALABILITÀ 10
BIBLIOGRAFIA 13

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Apertura

L'apertura è un obiettivo chiave dei sistemi distribuiti. Un sistema che fornisce componenti che vengono semplicemente utilizzati o incorporati in altri sistemi è noto come sistema distribuito aperto. Allo stesso tempo, un sistema open source includerà spesso componenti con origini esterne. Per essere aperti, i componenti devono rispettare le norme che specificano la sintassi e la semantica di ciò che tali componenti possono fare (cioè quale servizio forniscono).

L'utilizzo di un linguaggio di definizione dell'interfaccia (IDL) per definire i servizi tramite interfacce è una strategia comune. Molto spesso, le definizioni di interfaccia pubblicate in un IDL includono solo la sintassi dei servizi. In altre parole, sono abbastanza specifici sui nomi delle funzioni accessibili, nonché sui tipi di argomenti, sui valori restituiti, su eventuali eccezioni che potrebbero essere generate, ecc. Definire la semantica delle interfacce, o ciò che questi servizi realmente eseguiscono, è la parte più impegnativa. In realtà, questi criteri sono forniti in modo informale utilizzando il linguaggio naturale.

Una definizione di interfaccia, se fornita correttamente, consente a qualsiasi processo che richiede una determinata interfaccia di comunicare con un altro processo che offre tale interfaccia. Inoltre, consente a due parti indipendenti di sviluppare implementazioni completamente alternative di tali interfacce, risultando in due componenti distinti che funzionano in modo identico. Sono necessarie specifiche complete e imparziali. Completo indica che ogni dettaglio richiesto per l'implementazione è stato dichiarato. Tuttavia, molte definizioni di interfaccia sono solo parzialmente complete, rendendo necessaria l'aggiunta di informazioni specifiche dell'implementazione da parte di uno sviluppatore. L'idea che i requisiti debbano essere imparziali e non dettare come dovrebbe apparire un'attuazione è altrettanto cruciale. La completezza e la neutralità sono cruciali per l'interoperabilità e la portabilità.

Il grado in cui due implementazioni di sistemi o componenti di produttori diversi possono coesistere e funzionare insieme semplicemente dipendendo dai servizi reciproci come stabilito da una norma comune è noto come interoperabilità. Il grado in cui un'applicazione creata per un sistema distribuito **A** può essere eseguita, senza modifiche, su un altro sistema distribuito **B** che implementa le stesse interfacce di **A** è indicato come portabilità.

Dovrebbe essere semplice creare un sistema distribuito aperto con molti componenti, cosa che è un altro obiettivo molto importante (possibilmente da sviluppatori diversi). Inoltre, dovrebbe essere semplice sostituire i componenti obsoleti con quelli nuovi senza avere un impatto sui componenti che rimangono al loro posto. In altre parole, un sistema distribuito aperto dovrebbe essere in grado di crescere. Ad esempio, l'aggiunta di componenti che utilizzano un sistema operativo diverso o addirittura la

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

sostituzione dell'intero file system dovrebbe essere piuttosto semplice in un sistema espandibile. In pratica, si dimostra che molti sistemi distribuiti non sono così aperti come vorremmo e che è ancora necessario molto lavoro per assemblare parti diverse per creare un sistema distribuito.

Il codice sorgente effettivo per un componente può essere reso disponibile agli sviluppatori, che è un metodo per superare la mancanza di apertura. Questa strategia sta guadagnando popolarità, dando origine alle cosiddette iniziative open source in cui diverse persone lavorano insieme per migliorare e riparare i sistemi. Questo sistema è il più aperto possibile, ma se sia l'opzione migliore è argomento di discussione. È essenziale che il sistema sia impostato come un insieme di componenti relativamente piccoli e facilmente sostituibili o adattabili al fine di ottenere flessibilità nei sistemi distribuiti aperti. Ciò suggerisce che oltre a definire le interfacce di livello più alto, quelle percepite dagli utenti e dalle applicazioni, dovremmo anche definire le interfacce con i componenti di sistema sottostanti e spiegare come tali componenti interagiscono.

Molti sistemi più vecchi, e anche alcuni moderni, sono costruiti utilizzando un metodo monolitico, in cui le parti sono solo teoricamente divise ma sono effettivamente implementate come un unico, enorme programma. Con questo metodo, è difficile modificare o sostituire un componente senza interrompere l'intero sistema. Pertanto, i sistemi monolitici tendono ad essere chiusi piuttosto che aperti. Il requisito di modifica di un sistema distribuito è spesso causato da un componente che non fornisce i migliori criteri possibili per un particolare utente o applicazione.

Prendiamo come esempio il caching del browser Web. Esistono diversi criteri distinti che devono essere presi in considerazione.

- Memorizzazione: Dove devono essere memorizzati i dati nella cache? Nella maggior parte dei casi, sarà presente una cache in memoria adiacente all'archiviazione su disco. In quest'ultimo scenario, è necessario tenere conto della posizione precisa all'interno del file system locale.
- Esenzione: quali dati devono essere cancellati quando la cache è piena per fare spazio alle pagine appena recuperate?
- Condivisione: ogni browser utilizza la propria cache privata o più utenti possono condividere una singola cache?
- Aggiornamento: quando un browser controlla se i suoi dati memorizzati nella cache sono ancora aggiornati? Quando un browser può recuperare le pagine senza dover entrare in contatto con il sito Web di origine, le cache sono più efficaci. In questo caso, tuttavia, si corre il pericolo di restituire dati obsoleti. Inoltre, si tenga presente che le frequenze di aggiornamento dipendono in gran parte dal tipo di dati effettivamente memorizzati nella cache. Ad esempio, sebbene gli orari dei treni

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

cambino raramente, lo stesso non si può dire per i siti web che mostrano i valori correnti di mercato o, peggio ancora, le condizioni del traffico.

Abbiamo bisogno di una distinzione tra politica e meccanismo. Per la memorizzazione nella cache Web, ad esempio, un browser dovrebbe idealmente avere la possibilità di salvare solo i documenti, consentendo agli utenti di scegliere quali file conservare e per quanto tempo. Dando all'utente l'accesso a una vasta gamma di opzioni, questo può davvero essere fatto (dinamicamente). Un browser può anche fornire strumenti per collegare le policy che un utente ha creato come componente distinto se si fa un ulteriore passo avanti con questa idea.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Scalabilità

L'accesso a Internet in tutto il mondo è ormai una cosa normale per quasi tutte le persone. Inoltre, mentre in precedenza eravamo abituati a utilizzare computer desktop relativamente potenti per le applicazioni e l'archiviazione per ufficio, ora stiamo vedendo che tali applicazioni e servizi vengono collocati in quello che è noto come "il cloud", che è uno dei responsabili di un aumento dei dispositivi collegati in rete molto più piccoli, come ad esempio i tablet. Alla luce di ciò, la scalabilità è emersa come uno degli obiettivi chiave di progettazione per coloro che creano sistemi distribuiti.

La scalabilità di un sistema può essere valutata su almeno tre livelli distinti:

- Scalabilità delle dimensioni: quando si tratta di scalabilità, un sistema può essere semplicemente espanso per ospitare utenti e risorse aggiuntivi senza subire alcun degrado apprezzabile delle prestazioni.
- Scalabilità geografica: in un sistema geograficamente scalabile, gli utenti e le risorse possono essere distribuiti, ma i ritardi di comunicazione possono ancora essere evidenti.
- Scalabilità amministrativa: un sistema che può essere facilmente amministrato anche quando comprende diverse entità amministrative distinte è considerato scalabile dal punto di vista amministrativo.

Esaminiamo ciascuna di queste tre tipologie di scalabilità in modo più dettagliato. Per quanto riguarda la scalabilità delle dimensioni è necessario risolvere problemi di tipi molto diversi. Spesso ci imbattiamo in restrizioni di servizio centralizzate quando è necessario servire utenti o risorse aggiuntive, ma spesso per motivi molto diversi. Ad esempio, molti servizi nel sistema distribuito sono centralizzati nel senso che sono implementati utilizzando un singolo server che opera su un determinato computer. Un cluster di dispositivi strettamente connessi che sono fisicamente installati nella stessa posizione può ospitare un numero di server cooperanti in un ambiente più contemporaneo. Questo sistema ha un difetto apparente: di fronte a un aumento della domanda, il server, o l'insieme di server, può facilmente trasformarsi in un collo di bottiglia. Supponiamo che un servizio sia costruito su un singolo computer per dimostrare come ciò possa accadere. In tal caso, essere un collo di bottiglia ha fondamentalmente tre cause principali:

- La bassa capacità di calcolo delle CPU
- La rete tra l'utente e il servizio centralizzato;
- La capacità di archiviazione, inclusa la velocità di trasferimento I/O

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Si prenda in considerazione la potenza di calcolo, immaginando un servizio che calcola i percorsi migliori tenendo conto delle condizioni del traffico attuali. Non è difficile vedere che questo servizio può essere per lo più legato al calcolo e richiedere molte (decine di) secondi per elaborare una richiesta. Anche un sistema contemporaneo di fascia alta può in definitiva incorrere in problemi se è accessibile un solo computer e il numero di richieste supera una determinata soglia. Allo stesso modo, anche se per cause diverse, avremo problemi se abbiamo un servizio incentrato sull'I/O. Un motore di ricerca centrale mal costruito è un classico esempio. Si deve effettivamente abbinare una query di ricerca ad un intero set di dati quando si utilizzano query di ricerca basate sul contenuto, il che è un problema. Potremmo ancora riscontrare il problema di dover gestire una grande quantità di dati che supera la capacità di memoria primaria della macchina che esegue il servizio, anche con sofisticati algoritmi di indicizzazione. Di conseguenza, gli accessi al disco relativamente lenti e i trasferimenti di dati tra il disco e la memoria principale rappresenteranno una parte significativa del tempo di elaborazione. Man mano che il volume delle richieste cresce, la semplice aggiunta di dischi più veloci si rivelerà un'opzione insostenibile. Infine, una scalabilità inadeguata può anche essere il risultato della rete tra l'utente e il servizio.

Si consideri un'azienda di video on demand che deve trasmettere contenuti ad alta definizione a un numero di abbonati. Se un servizio stabilisce connessioni point-to-point con i propri consumatori, può raggiungere rapidamente la capacità di rete delle proprie linee di trasmissione in uscita poiché un flusso video potrebbe facilmente richiedere una larghezza di banda da 8 a 10 Mbps.

Ci sono problemi legati alla scalabilità geografica. Il fatto che molti sistemi distribuiti facciano ancora molto affidamento sulla comunicazione sincrona e siano stati costruiti per reti locali è uno dei motivi principali per cui la scalabilità è ancora impegnativa. Quando si utilizza questa modalità di comunicazione, la parte che richiede il servizio, che è più spesso nota come client, attende in uno stato sospeso fino a quando non riceve una risposta viene rimandata dal server che fornisce il servizio. Per essere più espliciti, vediamo spesso un modello di comunicazione che consiste in una moltitudine di scambi client-server, come a volte accade con le transazioni di database. Questa strategia ha spesso successo sulle LAN, dove la quantità di tempo necessaria a due computer per comunicare non supera in genere poche centinaia di microsecondi. Tuttavia, in un sistema che si estende su una vasta regione geografica, dobbiamo tenere presente che il tempo necessario ai processi per comunicare tra loro potrebbe essere di centinaia di millisecondi, che è tre ordini di grandezza più lento. La creazione di applicazioni che utilizzano la comunicazione sincrona in reti geografiche richiede molta attenzione (e non solo un po' di pazienza), in particolare con un ricco modello di interazione tra il client e il server.

La comunicazione su reti geografiche è intrinsecamente molto meno affidabile della comunicazione su reti locali, che è un altro problema che impedisce di raggiungere la scalabilità geografica. Oltre a questo,

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

dobbiamo anche fare i conti con la larghezza di banda limitata. Come risultato di questo impatto, le soluzioni progettate per l'uso in reti locali non sono necessariamente semplicemente trasferibili a sistemi WAN (Wide Area Network). Il video trasmesso in streaming online è un esempio tipico. Non è difficile mantenere un flusso coerente e rapido di fotogrammi video di buona qualità da un server multimediale a un display quando si utilizza una rete domestica, anche se sono disponibili solo connessioni wireless. Se si sposta semplicemente quel server in una posizione più remota e lo si collega al display tramite una connessione TCP convenzionale, senza dubbio fallirà. Non solo le restrizioni sulla larghezza di banda diventeranno immediatamente evidenti, ma anche mantenere lo stesso grado di affidabilità porterà rapidamente a problemi.

Quando i componenti sono distribuiti su una vasta area, possono sorgere una serie di complicazioni, una delle quali è il fatto che i sistemi ad ampia area hanno in genere pochissime strutture per la comunicazione multipunto. Le reti locali, d'altra parte, spesso forniscono strategie di trasmissione efficaci. Questo tipo di procedure si sono dimostrate incredibilmente utili nel processo di identificazione di componenti e servizi, che è fondamentale dal punto di vista della gestione. Quando si lavora con sistemi WAN (Wide Area Systems), è necessario creare servizi distinti, ad esempio servizi di denominazione e directory, a cui possono essere trasmesse query. Questi servizi di supporto, a loro volta, devono essere scalabili e in molte situazioni non sono disponibili risposte chiare.

Ultimo ma non meno importante, la questione di come scalare un sistema distribuito su più domini amministrativi indipendenti è una sfida che, in molti casi, rimane senza risposta. L'esistenza di regole contraddittorie in relazione all'uso (e al pagamento), alla gestione e alla sicurezza delle risorse è una questione significativa che deve essere risolta il prima possibile. Per fare un esempio, per un periodo di tempo significativo, i ricercatori hanno cercato modi per condividere le apparecchiature (a volte molto costose) che fanno parte di ciò che è noto come una griglia di calcolo (grid computing). Un sistema distribuito globale è costruito in queste griglie come una federazione di sistemi distribuiti locali. Questa configurazione consente a un programma che opera su un computer situato nell'organizzazione **A** di accedere direttamente alle risorse situate nell'organizzazione **B**. Gli utenti che svolgono le loro attività all'interno dello stesso dominio, ad esempio, hanno maggiori probabilità di essere in grado di riporre la loro fiducia nei diversi componenti di un sistema distribuito che si trovano all'interno di un singolo dominio. In situazioni come queste, l'amministrazione del sistema potrebbe aver testato e convalidato le applicazioni, nonché preso ulteriori precauzioni per garantire che questi componenti non possano essere alterati in alcun modo. Gli utenti hanno piena fiducia nei rispettivi amministratori di sistema. Tuttavia, in questo caso, la fiducia non cresce automaticamente oltre i confini del dominio.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Esistono due tipi distinti di misure di sicurezza che devono essere implementate nel caso in cui un sistema distribuito cresca in una nuova area. Per iniziare, il sistema distribuito deve dotarsi di difese contro gli attacchi dannosi provenienti dal nuovo dominio. Gli utenti del nuovo dominio, ad esempio, potevano avere accesso in lettura al file system solo nel vecchio dominio in cui era originariamente ospitato. Allo stesso modo, risorse come computer ad alte prestazioni o costosi non dovrebbero essere rese accessibili a persone che non sono autorizzate a usarle. In secondo luogo, il dominio appena creato deve prendere precauzioni per proteggersi da eventuali attacchi dannosi lanciati dal sistema distribuito. Il download di applicazioni nei browser Web, come gli applet, è un classico esempio di questo tipo di attività. Il nuovo dominio, in sostanza, non sa che tipo di comportamento anticiparsi da tale codice proveniente dall'esterno. La sfida è capire come mettere in atto tali restrizioni. Considerare le reti peer-to-peer contemporanee per la condivisione di file come un controesempio di sistemi distribuiti che si estendono su diversi domini amministrativi ma non sembrano soffrire di problemi di scalabilità amministrativa. Quando ciò si verifica, gli utenti finali devono solo installare un'applicazione che implementa funzionalità di ricerca e download distribuite e sono in grado di iniziare a scaricare i dati entro pochi minuti dal completamento dell'installazione. Altri esempi includono applicazioni peer-to-peer per la telefonia su Internet come Skype.

Gli utenti finali, a differenza degli organi amministrativi, sono quelli che lavorano insieme per garantire che i sistemi distribuiti continuino a funzionare correttamente. Questa è la caratteristica distintiva dei sistemi distribuiti. Le organizzazioni amministrative sottostanti come i provider di servizi Internet (ISP) possono, nella migliore delle ipotesi, controllare il traffico di rete creato da questi sistemi peer-to-peer, ma finora tali tentativi non hanno avuto particolare successo. I sistemi peer-to-peer sono uno dei principali fattori che contribuiscono alla congestione della rete.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Tecniche di scalabilità

Dopo aver esaminato alcuni dei problemi di scalabilità, ora siamo al punto in cui dobbiamo determinare come questi problemi possono essere risolti. Nella maggior parte dei casi, i problemi di scalabilità nei sistemi distribuiti si manifestano come problemi di prestazioni causati dalla capacità limitata dei server e della rete. La scalabilità verticale (scale up) è semplicemente il processo di aumento della capacità di un sistema, che può essere eseguito in diversi modi (ad esempio, aggiungendo più memoria, aggiornando le CPU o sostituendo i moduli di rete). Quando si tratta di scalabilità orizzontale (scale out), ovvero aumentare le dimensioni di un sistema distribuito aggiungendo efficacemente più computer, ci sono in realtà solo tre strategie che siamo in grado di implementare: nascondere le latenze di comunicazione, distribuire il lavoro e replicare i dati.

- Nascondere i ritardi nella trasmissione: la pratica di nascondere le latenze coinvolte nella comunicazione è applicabile nel contesto della scalabilità geografica. Il concetto alla base è semplice: rendere prioritario ridurre al minimo la quantità di tempo trascorso in attesa di risposte alle richieste fatte a servizi distanti. Ad esempio, se un utente di un computer distante ha richiesto un servizio, anziché attendere una risposta dal server, l'utente del computer remoto potrebbe svolgere altre attività produttive mentre la richiesta è in sospeso. In pratica, ciò comporta lo sviluppo dell'applicazione richiedente in modo tale che si impegni solo nella comunicazione asincrona. Nel caso in cui venga ricevuta una risposta, la domanda verrà terminata e verrà contattato un gestore dedicato al fine di eseguire l'azione precedentemente richiesta. La comunicazione asincrona può spesso essere utilizzata nei sistemi di elaborazione batch e nelle applicazioni parallele. Questi sono i tipi di programmi in cui è possibile pianificare l'esecuzione di singole attività mentre un altro processo attende il completamento della comunicazione. È inoltre possibile avviare un thread di controllo aggiuntivo per eseguire la richiesta come alternativa. Anche se causa il blocco dell'attesa per la risposta, la procedura può procedere con gli altri thread. Tuttavia, esiste un gran numero di applicazioni che non sono in grado di utilizzare la comunicazione asincrona in modo efficiente. Ad esempio, quando un utente invia una richiesta a un'applicazione interattiva, in genere non avrà niente di meglio da fare che attendere la risposta. In situazioni come queste, il modo più efficace per risolvere il problema è ridurre la quantità di comunicazione totale. Ciò può essere ottenuto, ad esempio, spostando parte del calcolo che viene tipicamente eseguito sul server al processo client che richiede il servizio. L'accesso alle banche dati tramite moduli è uno scenario atipico in cui questa strategia ha successo. L'invio di un messaggio separato per ogni

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

campo di un modulo e l'attesa di un riconoscimento da parte del server è un modo per completare la compilazione del modulo. Prima di accettare un elemento, il server può, ad esempio, fare un controllo per assicurarsi che non ci siano errori grammaticali. Sarebbe un approccio molto migliore inviare il codice per compilare il modulo e magari convalidare i dati al client e quindi fare in modo che il client restituisca un modulo completo. Questo metodo di spedizione del codice riceve un grande supporto da Internet, in particolare sotto forma di applet Java e Javascript.

- Processo di partizionamento e distribuzione: un'altra strategia di ridimensionamento essenziale è chiamata partizionamento e distribuzione e comporta la selezione di un componente, la suddivisione in parti più piccole e quindi la distribuzione di tali parti nel sistema. L'Internet Domain Name System è un eccellente esempio sia del partizionamento che della distribuzione. Per il DNS originale, lo spazio dei nomi DNS è organizzato gerarchicamente in una struttura ad albero di domini. Questi domini sono poi ulteriormente suddivisi in zone che non si sovrappongono l'una con l'altra. Un server singlename è responsabile della gestione dei nomi all'interno di ogni zona. Si può considerare ogni nome di percorso come il nome di un host da qualche parte su Internet. Per questo motivo, ogni nome di percorso è collegato all'indirizzo di rete dell'host a cui fa riferimento. La restituzione dell'indirizzo di rete dell'host collegato è ciò che conta davvero quando si desidera risolvere un nome. Si esamini il world wide web come ulteriore esempio. La maggior parte degli utenti ha l'impressione che il World Wide Web sia un enorme sistema informativo basato su documenti in cui ogni documento ha il proprio nome distintivo sotto forma di URL. Da un punto di vista logico, può anche dare l'impressione che ci sia un solo server. D'altra parte, il World Wide Web è fisicamente diviso e disperso su alcune centinaia di milioni di server, ognuno dei quali è responsabile di un certo numero di documenti Web. L'URL di un documento contiene una versione codificata del nome del server che sta elaborando la pagina. La capacità del World Wide Web di scalare fino alle dimensioni attuali è direttamente attribuibile al fatto che i documenti sono distribuiti in questo modo.
- Replicazione: spesso è una buona idea replicare realmente i componenti in un sistema distribuito. Ciò è dovuto al fatto che le difficoltà con la scalabilità si manifestano spesso sotto forma di deterioramento delle prestazioni. La replica non solo migliora la disponibilità, ma aiuta anche a distribuire il carico di lavoro tra i vari componenti, il che alla fine porta a prestazioni migliori. Avere una copia nelle vicinanze può anche nascondere una parte significativa dei problemi di latenza delle comunicazioni discussi in precedenza in sistemi geograficamente ampiamente dispersi. Sebbene distinguere tra i due possa essere difficile o addirittura impossibile a volte, la memorizzazione nella cache può essere considerata come un proprio sottoinsieme indipendente di replica. Quando un

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

client accede a una risorsa, la memorizzazione nella cache porta alla creazione di un duplicato di tale risorsa, che spesso si trova in prossimità del client che effettua la richiesta. La memorizzazione nella cache, d'altra parte, è una decisione presa da un client di una risorsa anziché dal proprietario della risorsa. Questo è in contrasto con la replica, che viene effettuata dal proprietario della risorsa. La memorizzazione nella cache e la replica presentano uno svantaggio significativo che potrebbe avere un impatto negativo sulla capacità di scalabilità del sistema. Poiché ora abbiamo diverse copie di una risorsa, se modifichiamo una copia, sarà distinta dalle altre. Come conseguenza diretta di ciò, la memorizzazione nella cache e la replica possono causare problemi di coerenza. Il grado in cui le discrepanze possono essere accettate dipende fortemente dall'utilizzazione della risorsa in questione. Ad esempio, un numero significativo di persone che utilizzano il World Wide Web accetta il fatto che il proprio browser Web restituirà una pagina memorizzata nella cache anche se la sua accuratezza non è stata verificata negli ultimi minuti. Tuttavia, vi sono anche molti casi in cui devono essere soddisfatte forti garanzie di coerenza, come nel caso delle borse e delle aste elettroniche. Questo è il caso in molte situazioni diverse. Una forte coerenza ha un grosso svantaggio, e cioè che qualsiasi modifica deve essere propagata istantaneamente a tutte le altre copie. Inoltre, se si verificano due aggiornamenti contemporaneamente, a volte è necessario che le modifiche vengano elaborate nello stesso ordine ovunque. Ciò crea un'ulteriore difficoltà di ordinamento globale. Potrebbe essere fisicamente impossibile combinare la coerenza con altre proprietà desiderabili come la disponibilità, aumentando ulteriormente la complessità dei problemi esistenti. Per questo motivo, la replica richiede spesso una sorta di meccanismo di sincronizzazione globale. Sfortunatamente, è estremamente difficile, se non impossibile, implementare tali tecniche in modo scalabile. Se non altro perché questo è il caso, è perché le latenze di rete hanno un vincolo naturale inferiore. Di conseguenza, la scalabilità tramite replica può comportare l'introduzione di nuove soluzioni che non sono intrinsecamente scalabili.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- Blair G. and Stefani J.-B. Open Distributed Processing and Multimedia. Addison-Wesley, Reading, MA., 1998.
- Neuman B. Scale in Distributed Systems. In Casavant T. and Singhal M., editors, Readings in Distributed Computing Systems, pages 463–489. IEEE Computer Society Press, Los Alamitos, CA., 1994.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).